

UNIT – 1

INTRODUCTION TO WEB TECHNOLOGY AND WEB PAGE DESIGNING

1. Introduction to Web Development

Web Development is the process of creating websites and web applications that run on the Internet or an intranet. It involves designing web pages, developing server-side applications, managing databases, and deploying websites for users.

Types of Web Development

1. Front-End Development

Front-End Development focuses on the user interface and user experience of a website.

Technologies Used

- HTML
- CSS
- JavaScript
- React
- Angular
- Vue.js

2. Back-End Development

Back-End Development handles business logic, databases, and server-side processing.

Technologies Used

- Node.js
- Java
- PHP
- Python
- .NET

3. Full Stack Development

A Full Stack Developer works on both Front-End and Back-End development.

2. Web Development Strategies

Static Websites

A static website contains fixed content that remains the same for all users.

Advantages

- Easy to develop
- Fast loading
- Low hosting cost

Disadvantages

- Difficult to update frequently
- Limited interactivity

Examples

- Personal portfolios
- Company profile websites

Dynamic Websites

Dynamic websites generate content according to user requests.

Advantages

- Interactive
- Database support
- Personalized content

Examples

- Facebook
- Amazon
- Instagram

3. History of Web and Internet

Internet

The Internet is a worldwide network connecting millions of computers.

Evolution of Internet

Year	Development
1969	ARPANET established
1983	TCP/IP adopted
1989	Tim Berners-Lee proposed WWW

Year	Development
1991	First Website launched
1993	Mosaic Browser released
1998	Google founded
Present Cloud and AI-based Web Applications	

World Wide Web (WWW)

The World Wide Web is a collection of interconnected web pages accessible through the Internet.

Components of WWW

- Web Browser
- Web Server
- URL
- HTTP Protocol

4. Protocols Governing the Web

A protocol is a set of rules used for communication over a network.

HTTP (HyperText Transfer Protocol)

Used for transferring web pages between browser and server.

Example:

`http://www.example.com`

Features

- Stateless protocol
- Request-response model

HTTPS (HyperText Transfer Protocol Secure)

Secure version of HTTP using SSL/TLS encryption.

Example:

`https://www.google.com`

Advantages

- Data confidentiality
- Authentication

- Data integrity

FTP (File Transfer Protocol)

Used for transferring files between computers over a network.

SMTP (Simple Mail Transfer Protocol)

Used for sending emails.

POP3 and IMAP

Used for receiving and managing emails.

TCP/IP

Foundation protocol suite of the Internet.

TCP

Provides reliable communication.

IP

Provides addressing and routing.

5. Writing Web Projects

Web project development follows a systematic process.

Phases

1. Requirement Analysis

Understanding client requirements.

2. Design

Preparing page layouts and architecture.

3. Development

Coding the application.

4. Testing

Finding and fixing errors.

5. Deployment

Making the application available to users.

6. Maintenance

Updating and improving the application.

6. Connecting to the Internet

Requirements

- Computer or Mobile Device
- Network Interface Card (NIC)
- Modem/Router
- Internet Service Provider (ISP)

Types of Internet Connections

Dial-Up Connection

Uses telephone lines.

Broadband Connection

Provides high-speed internet.

Fiber Optic Connection

Uses optical fibers for very high speed.

Wireless Connection

Includes Wi-Fi and Mobile Networks.

7. Internet Services and Tools

Internet Services

E-Mail

Electronic communication over the Internet.

Examples:

- Gmail
- Outlook

World Wide Web

Accessing web pages and websites.

FTP

File sharing and transfer.

VoIP

Voice communication over Internet.

Examples:

- Skype
- WhatsApp Calling

Cloud Services

Online storage and computing.

Examples:

- Google Drive
- OneDrive

Internet Tools

Web Browsers

Examples:

- Chrome
- Firefox
- Edge

Search Engines

Examples:

- Google
- Bing

FTP Clients

Examples:

- FileZilla
- WinSCP

8. Client-Server Computing

Client-Server Computing is a distributed computing model in which clients request services and servers provide services.

Client

A software application or device that requests resources.

Examples:

- Web Browser
- Mobile Application

Server

A computer system that provides services.

Examples:

- Web Server
- Database Server

Architecture

```
Client ---- Request ----> Server
Client <--- Response ---- Server
```

Advantages

- Centralized management
- Better security
- Resource sharing

9. HTML (HyperText Markup Language)

HTML is the standard markup language used to create web pages.

Basic Structure

```
<!DOCTYPE html>
<html>
<head>
```

```
<title>My First Web Page</title>
</head>
<body>
<h1>Welcome to HTML</h1>
</body>
</html>
```

10. HTML Lists

Ordered List

```
<ol>
<li>C</li>
<li>C++</li>
<li>Java</li>
</ol>
```

Unordered List

```
<ul>
<li>Apple</li>
<li>Mango</li>
<li>Orange</li>
</ul>
```

Description List

```
<dl>
<dt>HTML</dt>
<dd>HyperText Markup Language</dd>
</dl>
```

11. HTML Tables

```
<table border="1">
<tr>
<th>Name</th>
<th>Marks</th>
</tr>
<tr>
<td>Rahul</td>
<td>90</td>
</tr>
</table>
```

Table Tags

- table
- tr
- th
- td
- caption

12. HTML Images

```

```

Attributes

- src
- alt
- width
- height

13. HTML Frames and IFrames

Frame

Frames divide the browser window into multiple sections.

```
<frameset cols="30%,70%">
<frame src="menu.html">
<frame src="content.html">
</frameset>
```

IFrame

```
<iframe
src="https://www.google.com"
width="500"
height="300">
</iframe>
```

14. HTML Forms

Forms are used to collect user input.

```
<form>
Name:
<input type="text">

Password:
<input type="password">

<input type="submit">
</form>
```

Common Controls

- Text Box
- Password Box
- Radio Button

- Check Box
- Drop-down List
- Submit Button
- Reset Button

15. XML (Extensible Markup Language)

XML is a markup language used for storing and transporting data.

Example

```
<student>
  <name>Rahul</name>
  <roll>101</roll>
</student>
```

Features

- Self-descriptive
- Platform Independent
- Extensible
- Human Readable

16. Document Type Definition (DTD)

DTD defines the structure and legal elements of an XML document.

```
<!DOCTYPE student [
<!ELEMENT student (name,roll)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT roll (#PCDATA)>
]>
```

17. XML Schema (XSD)

XML Schema provides a more powerful way to define XML structures.

Advantages

- Supports data types
- More secure
- XML based

18. XML Object Model

XML documents are represented as a tree structure.

```
Student
|
```

```
+-- Name
|
+-- Roll
```

19. Presenting and Using XML

XML can be displayed using:

- CSS
- HTML
- JavaScript
- XSLT

Example:

```
<?xml-stylesheet
type="text/css"
href="style.css"?>
```

20. XML Processors

XML Processors read and validate XML documents.

Types

Validating Processor

Checks XML against DTD/XSD.

Non-Validating Processor

Checks only syntax.

21. DOM (Document Object Model)

DOM loads the complete XML document into memory and represents it as a tree structure.

Advantages

- Easy navigation
- Easy modification

Disadvantages

- High memory consumption

22. SAX (Simple API for XML)

SAX is an event-driven XML parser.

Advantages

- Fast
- Memory efficient

Disadvantages

- Difficult to modify data

DOM vs SAX

Feature	DOM	SAX
Loading	Entire Document	Sequential
Memory Usage	High	Low
Speed	Moderate	Fast
Modification	Easy	Difficult
Suitable For	Small XML Files	Large XML Files

UNIT – 2

CASCADING STYLE SHEETS (CSS)

1. Introduction to CSS

What is CSS?

CSS (Cascading Style Sheets) is a style sheet language used to control the presentation and appearance of HTML documents.

CSS separates content from presentation, making websites easier to maintain and design.

Advantages of CSS

- Improves website appearance
- Reduces code duplication
- Faster page loading
- Easy maintenance
- Responsive design support

Syntax of CSS

```
selector {
```

```
    property: value;
}
```

Example:

```
h1 {
  color: blue;
  font-size: 30px;
}
```

2. Creating Style Sheets

CSS can be applied in three ways.

2.1 Inline CSS

Applied directly within an HTML tag.

```
<h1 style="color:red;">Welcome</h1>
```

Advantages

- Quick styling

Disadvantages

- Difficult to maintain

2.2 Internal CSS

Defined inside the `<style>` tag.

```
<head>
<style>
h1{
  color:blue;
}
</style>
</head>
```

2.3 External CSS

Stored in a separate file.

style.css

```
h1{
  color:green;
}
```

HTML File

```
<link rel="stylesheet" href="style.css">
```

Advantages

- Reusable
- Easy maintenance
- Professional approach

3. CSS Properties

A CSS property specifies how an HTML element should appear.

General Syntax

```
selector{  
    property:value;  
}
```

Examples:

```
color:red;  
background-color:yellow;  
font-size:20px;
```

4. CSS Styling

4.1 Background Properties

Used to style page backgrounds.

Background Color

```
body{  
    background-color:lightblue;  
}
```

Background Image

```
body{  
    background-image:url("bg.jpg");  
}
```

Background Repeat

```
background-repeat:no-repeat;
```

Background Position

```
background-position:center;
```

4.2 Text Formatting

Text Color

```
p{  
    color:red;  
}
```

Text Alignment

```
text-align:center;
```

Text Decoration

```
text-decoration:underline;
```

Text Transformation

```
text-transform:uppercase;
```

Line Height

```
line-height:1.5;
```

4.3 Controlling Fonts

Font Family

```
font-family:Arial;
```

Font Size

```
font-size:20px;
```

Font Style

```
font-style:italic;
```

Font Weight

```
font-weight:bold;
```

Shorthand Property

```
font:bold 18px Arial;
```

5. Working with Block Elements

Block Elements

Occupy the entire available width.

Examples:

- `div`
- `p`
- `h1-h6`
- `section`

```
div{  
    background-color:lightgray;  
}
```

Inline Elements

Occupy only required space.

Examples:

- `span`
- `a`
- `img`
- `strong`

```
span{  
    color:red;  
}
```

6. Working with Lists

Unordered List Styling

```
ul{  
    list-style-type:square;  
}
```

Ordered List Styling

```
ol{  
    list-style-type:upper-roman;  
}
```

Custom List Image

```
ul{  
    list-style-image:url(icon.png);  
}
```

7. Working with Tables

```
table{
    border-collapse:collapse;
}

th,td{
    border:1px solid black;
    padding:10px;
}
```

Table Properties

- border
- border-collapse
- padding
- width
- text-align

8. CSS ID and Class

ID Selector

Used for a unique element.

```
<p id="intro">Hello</p>
#intro{
    color:red;
}
```

Characteristics

- Unique
- Starts with #

Class Selector

Used for multiple elements.

```
<p class="highlight">Text</p>
.highlight{
    color:blue;
}
```

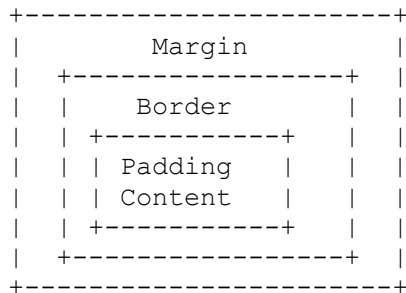
Characteristics

- Reusable
- Starts with .

9. CSS Box Model

Every HTML element is represented as a rectangular box.

Components



9.1 Border Properties

Border Width

```
border-width:2px;
```

Border Style

```
border-style:solid;
```

Border Color

```
border-color:red;
```

Shorthand

```
border:2px solid blue;
```

9.2 Padding Properties

Padding creates space inside the border.

```
padding:20px;
```

Individual Sides:

```
padding-top:10px;
padding-right:20px;
padding-bottom:10px;
padding-left:20px;
```

9.3 Margin Properties

Margin creates space outside the border.

```
margin:20px;
```

Individual Sides:

```
margin-top:10px;  
margin-right:20px;  
margin-bottom:10px;  
margin-left:20px;
```

ADVANCED CSS

10. Grouping Selectors

Apply same style to multiple elements.

```
h1,h2,h3{  
    color:blue;  
}
```

11. Dimension Properties

Width

```
width:300px;
```

Height

```
height:200px;
```

Maximum Width

```
max-width:800px;
```

Minimum Height

```
min-height:100px;
```

12. Display Property

Controls element display behavior.

Block

```
display:block;
```

Inline

```
display:inline;
```

Inline Block

```
display:inline-block;
```

None

```
display:none;
```

13. Positioning

Static

Default positioning.

```
position:static;
```

Relative

```
position:relative;  
top:20px;
```

Absolute

```
position:absolute;  
left:50px;
```

Fixed

```
position:fixed;  
bottom:0;
```

Sticky

```
position:sticky;  
top:0;
```

14. Floating Elements

Left Float

```
float:left;
```

Right Float

```
float:right;
```

Clear Float

```
clear:both;
```

15. Alignment

Horizontal Centering

```
margin:auto;
```

Text Alignment

```
text-align:center;
```

Vertical Alignment

```
vertical-align:middle;
```

16. Pseudo Classes

Pseudo classes define special states of elements.

Link States

```
a:link{
    color:blue;
}

a:visited{
    color:purple;
}

a:hover{
    color:red;
}

a:active{
    color:green;
}
```

First Child

```
p:first-child{
    color:red;
}
```

17. Navigation Bar

Horizontal Navigation Bar

```
<ul>
<li><a href="#">Home</a></li>
<li><a href="#">About</a></li>
<li><a href="#">Contact</a></li>
</ul>
ul{
```

```

    list-style:none;
}

li{
    display:inline-block;
    padding:10px;
}

```

18. Image Sprites

An Image Sprite combines multiple images into one image file.

Advantages

- Faster loading
- Reduced HTTP requests
- Better performance

Example:

```

.icon{
    background:url(sprite.png);
}

```

19. Attribute Selectors

Select elements based on attributes.

Example

```

input[type="text"]{
    background-color:lightyellow;
}

```

Multiple Examples

```

a[target="_blank"]{
    color:red;
}

```

20. CSS Colors

Color Name

```

color:red;

```

RGB

```

color:rgb(255,0,0);

```

HEX

```
color:#ff0000;
```

HSL

```
color:hsl(0,100%,50%);
```

21. Creating Page Layouts and Site Designs

Two Column Layout

```
<div class="container">
<div class="sidebar">Sidebar</div>
<div class="content">Content</div>
</div>
.container{
    display: flex;
}

.sidebar{
    width: 30%;
}

.content{
    width: 70%;
}
```

Modern Layout Using Flexbox

```
.container{
    display: flex;
    justify-content: space-between;
    align-items: center;
}
```

Grid Layout

```
.container{
    display: grid;
    grid-template-columns: 1fr 1fr 1fr;
    gap: 20px;
}
```

Advantages

- Responsive design
- Easy alignment
- Flexible layouts

UNIT – 3

SCRIPTING AND NETWORKING

PART A: SCRIPTING (JAVASCRIPT)

1. Introduction to JavaScript

What is JavaScript?

JavaScript is a lightweight, interpreted, object-based scripting language used to create dynamic and interactive web pages.

JavaScript was developed by Brendan Eich in 1995 and is one of the core technologies of the World Wide Web along with HTML and CSS.

Features of JavaScript

- Lightweight scripting language
- Object-oriented
- Event-driven
- Platform independent
- Case sensitive
- Supports client-side and server-side programming

Applications of JavaScript

- Form validation
- Dynamic content updates
- Interactive web pages
- Animations
- AJAX applications
- Web-based games

2. JavaScript in HTML

Internal JavaScript

```
<!DOCTYPE html>
<html>
<head>
<script>
    document.write("Welcome to JavaScript");
</script>
</head>
<body>
</body>
</html>
```

External JavaScript

```
<script src="script.js"></script>
```

3. JavaScript Variables

Variables are used to store data.

Using var

```
var name = "Rahul";
```

Using let

```
let age = 20;
```

Using const

```
const PI = 3.14159;
```

4. Data Types in JavaScript

Primitive Data Types

- Number
- String
- Boolean
- Undefined
- Null
- BigInt
- Symbol

Example

```
let marks = 95;  
let name = "Rahul";  
let result = true;
```

5. JavaScript Operators

Arithmetic Operators

```
let a = 10;  
let b = 5;  
  
console.log(a+b);  
console.log(a-b);  
console.log(a*b);  
console.log(a/b);
```

Relational Operators

```
a > b
a < b
a == b
a != b
```

Logical Operators

```
&&
||
!
```

6. JavaScript Statements

Statements are instructions executed by JavaScript.

Conditional Statements

if Statement

```
let age = 18;

if(age >= 18){
    document.write("Eligible to Vote");
}
```

if-else Statement

```
if(age >= 18){
    document.write("Eligible");
}
else{
    document.write("Not Eligible");
}
```

switch Statement

```
let day = 2;

switch(day){
    case 1:
        document.write("Monday");
        break;

    case 2:
        document.write("Tuesday");
        break;

    default:
        document.write("Invalid Day");
}
```

7. Looping Statements

for Loop

```
for(let i=1;i<=5;i++){  
    document.write(i);  
}
```

while Loop

```
let i=1;  
  
while(i<=5){  
    document.write(i);  
    i++;  
}
```

do-while Loop

```
let i=1;  
  
do{  
    document.write(i);  
    i++;  
}while(i<=5);
```

8. JavaScript Functions

Functions are reusable blocks of code.

Function Declaration

```
function add(a,b){  
    return a+b;  
}  
  
document.write(add(10,20));
```

Function Expression

```
const square = function(x){  
    return x*x;  
};
```

Arrow Function

```
const cube = (x)=>{  
    return x*x*x;  
};
```

9. JavaScript Objects

Objects store data as key-value pairs.

Creating an Object

```
let student = {  
  name:"Rahul",  
  roll:101,  
  course:"CSE"  
};
```

Accessing Object Properties

```
document.write(student.name);  
document.write(student.roll);
```

Adding Properties

```
student.city = "Delhi";
```

10. JavaScript Arrays

An array stores multiple values in a single variable.

```
let subjects = ["Java", "Python", "C++"];
```

Accessing Elements

```
document.write(subjects[0]);
```

Looping Through Array

```
subjects.forEach(function(item) {  
  document.write(item);  
});
```

11. JavaScript Document Object Model (DOM)

DOM represents an HTML document as a tree structure.

Accessing Elements

```
document.getElementById("demo");
```

Changing Content

```
document.getElementById("demo").innerHTML =  
"Welcome Students";
```

Changing Style

```
document.getElementById("demo").style.color =  
"red";
```

12. JavaScript Forms

Forms are commonly validated using JavaScript.

Example

```
<form onsubmit="return validate() ">

<input type="text" id="name">

<input type="submit">

</form>
function validate(){

    let name =
    document.getElementById("name").value;

    if(name==""){
        alert("Name Required");
        return false;
    }

    return true;
}
```

13. JavaScript Events

Events occur when users interact with web pages.

Common Events

Event	Description
onclick	Mouse click
ondblclick	Double click
onkeyup	Key release
onmouseover	Mouse hover
onsubmit	Form submission
onload	Page load

Example

```
<button onclick="showMessage() ">
Click
</button>
function showMessage(){
    alert("Hello Students");
}
```

14. Introduction to AJAX

What is AJAX?

AJAX stands for:

Asynchronous JavaScript And XML

AJAX allows web pages to communicate with servers without reloading the entire page.

Benefits

- Faster response
- Better user experience
- Reduced server load
- Partial page updates

AJAX Workflow

```
Browser
  |
AJAX Request
  |
Server
  |
Response
  |
Update Web Page
```

Example Using Fetch API

```
fetch("students.json")
  .then(response => response.json())
  .then(data => console.log(data));
```

PART B: NETWORKING

15. Introduction to Networking

Networking is the process of connecting computers and devices to share information and resources.

Advantages

- Resource sharing
- Communication
- Remote access
- File sharing

16. Internet Addressing

Every device connected to the Internet is identified by an IP Address.

IPv4 Address

192.168.1.1

IPv6 Address

2001:0db8:85a3::8a2e:0370:7334

Domain Name

Example:

www.google.com

Domain names are translated into IP addresses using DNS.

17. InetAddress Class (Java)

The InetAddress class is used to represent IP addresses in Java.

Package

```
import java.net.*;
```

Example

```
InetAddress ip =  
InetAddress.getLocalHost();  
  
System.out.println(ip);
```

18. Factory Methods of InetAddress

Factory methods create InetAddress objects.

getLocalHost()

```
InetAddress ip =  
InetAddress.getLocalHost();
```

getByName()

```
InetAddress ip =  
InetAddress.getByName("www.google.com");
```

getAllByName()

```
InetAddress[] ip =
```

```
InetAddress.getAllByName("www.google.com");
```

19. Instance Methods of InetAddress

getHostName()

```
System.out.println(ip.getHostName());
```

getHostAddress()

```
System.out.println(ip.getHostAddress());
```

isReachable()

```
System.out.println(ip.isReachable(1000));
```

20. URL (Uniform Resource Locator)

A URL specifies the location of a resource on the Internet.

Structure

```
protocol://host:port/path
```

Example

```
https://www.google.com/search
```

Components

Component	Example
-----------	---------

Protocol	https
Host	google.com
Port	443
Path	/search

21. URL Class in Java

```
import java.net.*;

URL url =
new URL("https://www.google.com");

System.out.println(url.getProtocol());
System.out.println(url.getHost());
System.out.println(url.getPort());
```

22. URLConnection

URLConnection provides communication with resources identified by a URL.

Example

```
URL url =
new URL("https://www.google.com");

URLConnection con =
url.openConnection();

System.out.println(con.getContentType());
```

23. TCP/IP Client Sockets

A socket represents an endpoint of communication.

Client Socket Example

```
import java.net.*;
import java.io.*;

Socket socket =
new Socket("localhost",5000);

PrintWriter out =
new PrintWriter(
socket.getOutputStream(),
true);

out.println("Hello Server");

socket.close();
```

24. TCP/IP Server Sockets

A ServerSocket waits for client connections.

Server Example

```
import java.net.*;

ServerSocket server =
new ServerSocket(5000);

Socket socket =
server.accept();

System.out.println(
"Client Connected");
```

25. Socket Communication Architecture

```

Client
 |
Socket
 |
Internet
 |
ServerSocket
 |
Server

```

Advantages of TCP

- Reliable
- Ordered delivery
- Error checking

26. Datagram Communication (UDP)

UDP uses datagrams instead of connections.

Features

- Connectionless
- Faster than TCP
- Less reliable

Datagram Packet Example

```

DatagramSocket socket =
new DatagramSocket();

byte[] data =
"Hello".getBytes();

DatagramPacket packet =
new DatagramPacket(
data,
data.length,
InetAddress.getByName("localhost"),
5000);

socket.send(packet);

```

TCP vs UDP

Feature	TCP	UDP
Connection	Required	Not Required
Reliability	High	Low
Speed	Slower	Faster
Error Checking	Yes	Limited
Examples	HTTP, FTP	DNS, Streaming

